LIST OF EXPERIMENTS

Sl. No.	Date	Name of the Experiment	Page No.	Marks
1	13/08/25	Introduction	1-5	5
2	13/08/25	Develop an embedded C program to blink an internal LED of Nucleo F411RE board	6	5
3	20/08/25	Develop an embedded C program to blink an external LED & send pulses to CRO	7-8	5
4	10/09/25	Design & implement an edge application on STM32 that processes a locally generated counter value.	9-10	5
5	10/09/25	Design & implement an edge application on STM32 that enables UART based text messaging b/w board & serial terminal	11-12	5
6	15/10/25	To interface a DHT11 temperature & humidity sensor with Raspberry Pi.	13-15	5
7	15/10/25	To interface an audio sensor with an STM32 nucleo board via ADC	16-17	5
8	22/10/25	To interface ultrasonic sensor with Raspberry Pi	18-19	5
9				
10				

Experiment - 02

Aim: To develop an embedded C program to blink an external LED connected to PB5 of STM32 and send pulses to oscillator.

Components required: PC with STM32 cube IDE software, STM32 hardware kit & wires.

Procedure:

* Go to File → New → STM32 Project.

* Target selection → Board selection → commercial part number → Nucleo F411RE → Select Board → Next.

* File → Project name: LED_PB5 → target language: C → Finish.

* GPIO pin configation: Click on PB5 → GPIO output → generate code.

* In main.c file go to while (1) loop and write the code.

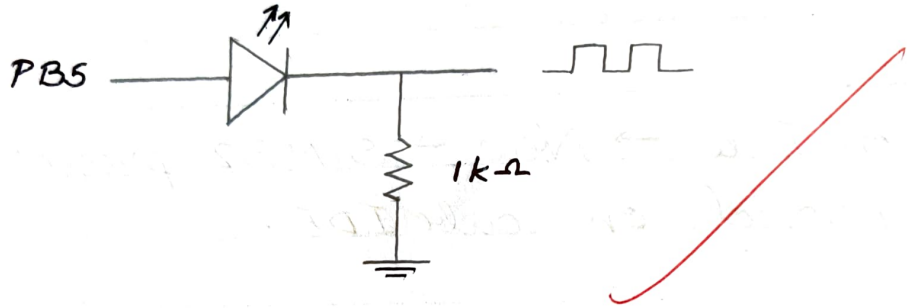
* Click on the build icon to compile the code.

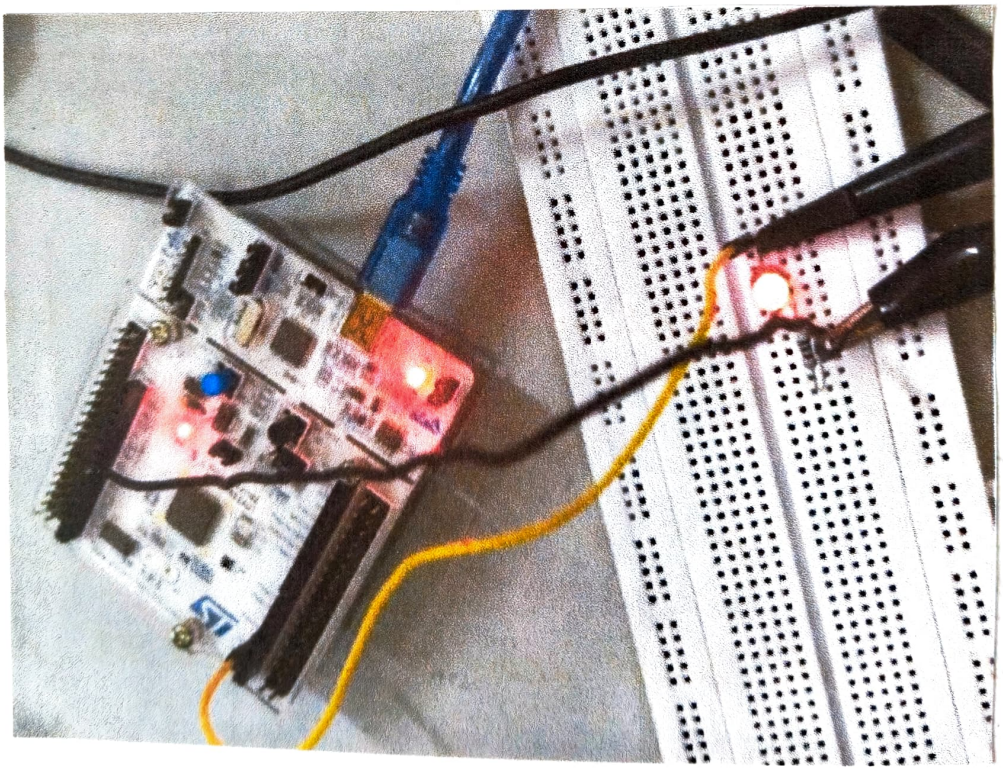
* Connect your STM32 board to the PC.

* Click the STM32 board to the PC.

Program:

```
HAL_GPIO_WritePin (GPIOB, GPIO_PIN_5, 0);
HAL_Delay (300);
HAL_GPIO_WritePin (GPIOB, GPIO_PIN_5, 1);
HAL_Delay (300);
```





d d m m y y y y

* Click the debug icon → Run the program.

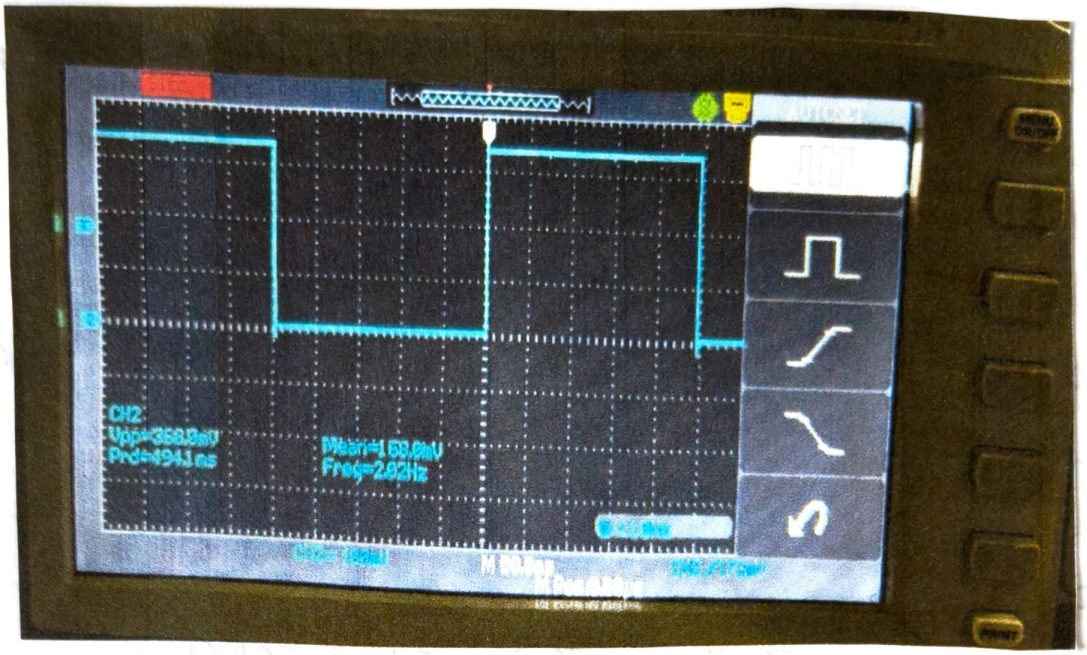
Result:

The external LED on PB5 blinks at a fixed interval and pulse signals are successfully transmitted to the oscillator.

(5)

22/8/25

CRO output.



Experiment - 03

Aim: To design and implement an edge application in STM32 that processor are totally generated counter values and transmitted via UART to serial terminal.

Components: PC with STM32 cubeIDE software, STM32 hardware kit and wires.

Procedure:

* Open STM32 cubeIDE → Go to File → New → STM32 project.

* Select your board → use the Board selector tab → choose your STM32 board → Click Next.

* Enter the project name and click Next.

* Enable UART (USART 2)
  - Go to pinout and configuration.
  - Select USART 2 → mode : Asynchronous.
  - Tx and Rx pin will be auto-assigned.

* Click on generate code.

* In main.c locate the while(1) loop and write the code to generate and transmit counter value via UART.

* Click on Build to compile.

Program:

```
#include <main.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main(void)
{
  char message[40] = {'\0'};
  int counter = 0;

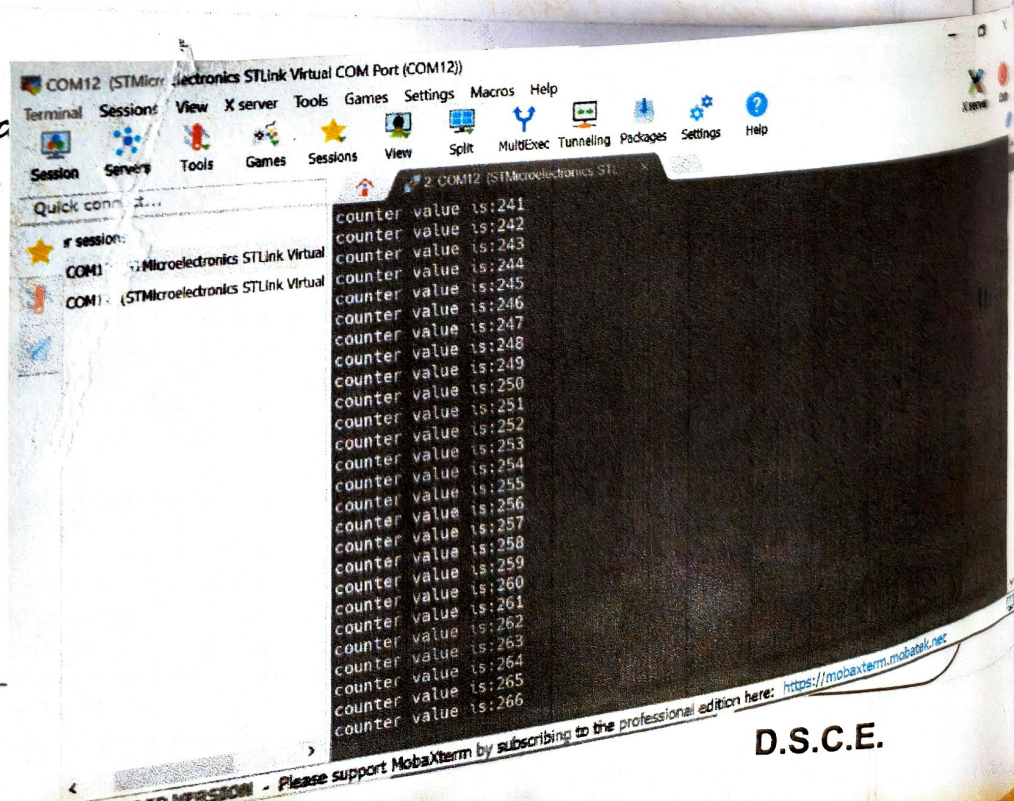
while (1)
{
  SPrintf(message, "Counter value is: %d \n\r", counter);
HAL_UART_Transmit(&huart2, (uint8_t*)message,
                  strlen(message), 100);

HAL_Delay(500);
counter++;
```

Output:

Serial Termina

```
Counter value is:
Counter value is:
.
.
.
Counter value is:
```



Program:

```
#include <main.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main(void)
{
  char message[40] = {'\0'};
  int counter = 0;

while (1)
{
  SPrintf(message, "Counter value is: %d \n\r", coun
  HAL_UART_Transmit (&huart2, (unit8_t*) mess
                     strlen(message), 100);

  HAL_Delay (500);
  counter ++;
```

Output:

Serial Terminal:

```
Counter value is: 44
Counter value is: 45
  .
  .
  .
Counter value is: n
```

d d m m y y y y

* Connect the STM32 Board to your PC using USB.

* To open serial terminal click on MobaXterm.
  - Select the com Serial port
  - speed - 11520

* Click the debug icon in cube IDE → then click OK.

* Click the Run to execute the program.

* Observe the output.

Result:

The STM32 board successfully generates a counter value and transmits it through UART to the serial terminal continuously.

(5)

20/8/3

10/09/2025

Experiment - 04

Aim: Design and implementation on edge application on STM32 that enables UART based text messaging between the board, a serial terminal allowing local processing of received message dynamic control of system behavior based on user input.

Components: STM32 cube IDE software, STM32 hardware kit and wires.

Procedure:

* Open STM32 cube IDE → Go to File → New → STM32 project.

* Select Board → use the Board selector tab → choose your STM32 board → click Next.

* Enter the project name and click on Next.

* Enable UART communication.
  - In the pinout & configuration tab enable USART2.
  - Mode: Asynchronous.
  - TX → PA2 ; RX → PA3.

* Set UART parameters
  - Baud rate : 9600
  - Data bits : 8

Program:

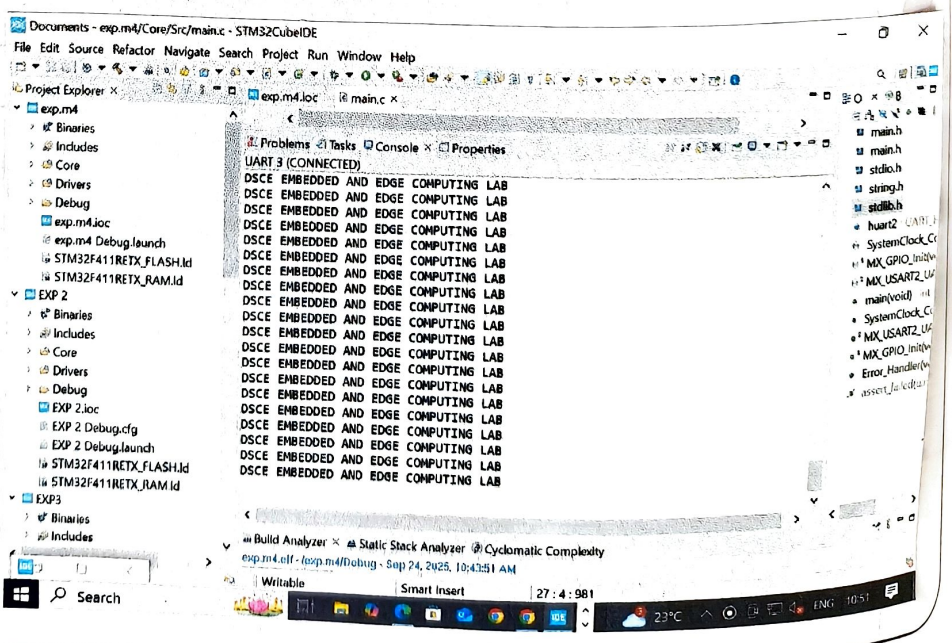
```
#include <main.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main(void)
{
    char message[40] = {'\0'};
}

while (1)
{
  strcpy (message "Hello World ");
  HAL_UART_Transmit (&huart2, (uint8_t *) message,
                     strlen(message), 100);
  HAL_Delay (500);
}
```

Output:

Hello World
Hello World
.
.
.
.
.
Hello World



d d m m y y y y

* Click on generate code.

* Open main.c and click on build to compile..

* Connect the STM32 board to PC using USB.

* Open MobaXterm → Select com serial port.

* Start debugging in cube IDE.

* Run the program & see the output.

Result:

Text typed in MobaXterm is received and displayed back by the STM32 board. This confirms successful UART communication and real time message processing.

5

5 10/9/25

15 Nov 2025

Experiment - 05

Aim: To interface a DHT-11 temperature & humidity sensor with Raspberry Pi integrated on IoT development board (ARM/IOT) and upload the sensed data to ThingSpeak cloud platform for remote monitoring using Python.

Components required: IoT dev board, Raspberry Pi, DHT11 sensor, ethernet/wi-fi, HDMI cable & display, & keyboard, mouse, Thingspeak cloud account and python with libraries.

Procedure:

i. Hardware Setup:

* Use the IoT development board which contains on-board Raspberry Pi connected internally to various sensors and modules.
* Connect the HDMI cable from the Raspberry Pi to a display monitor.
* The DHT 11 sensor is pre-connected to one of the Raspberry Pi GPIO pins.

ii. ThingSpeak configuration:

* Open ThingSpeak.com and create a new channel with the following fields:

Field 1: Temperature

Field 2: Humidity.

Program:

```
import time
import math
import Adafruit_DHT as dht

myAPI = 'CD7F6A23EUX518A'
baseURL = 'https://api.thingspeak.com'

GPIO.setmode(GPIO.BCM)

sensor = dht.DHT11
dht11_pin = 4

temp = 0.0
humidity = 0.0

print("IOT Development Kit")

while True:
    try:
        print("Scanning for sensor data.....")
        time.sleep(2)
        humidity, temp = dht.read_retry(sensor, dht11_pin)
        if math.isnan(temp) == False and math.isnan(humidity)
            print("Temperature = %.02f C "%(temp))==False:
            print("Humidity = %.02f %% "%(humidity))

    conn = urllib.urlopen(baseURL + &field1 = %s & field2 =%s'
                          %(temp, humidity))
```

d d m m y y y y

* Copy the write API key for use in the Python code.

iii. Python Program Development:
* Open the Thonny IDE or terminal on the Raspberry Pi display.
* Create a new file named IoT_cloud_DHT.py.
* Define the API key, base URL and sensore type.

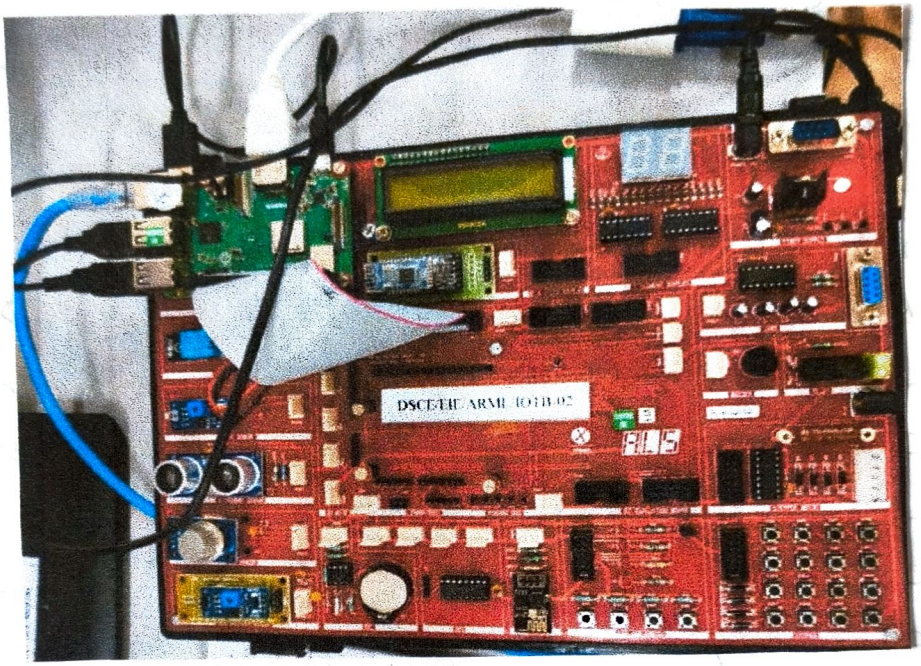
iv. Program Logic:
* Continuously read temperature and humidity.
* If readings are valid, send them to the ThingSpeak cloud using URL.

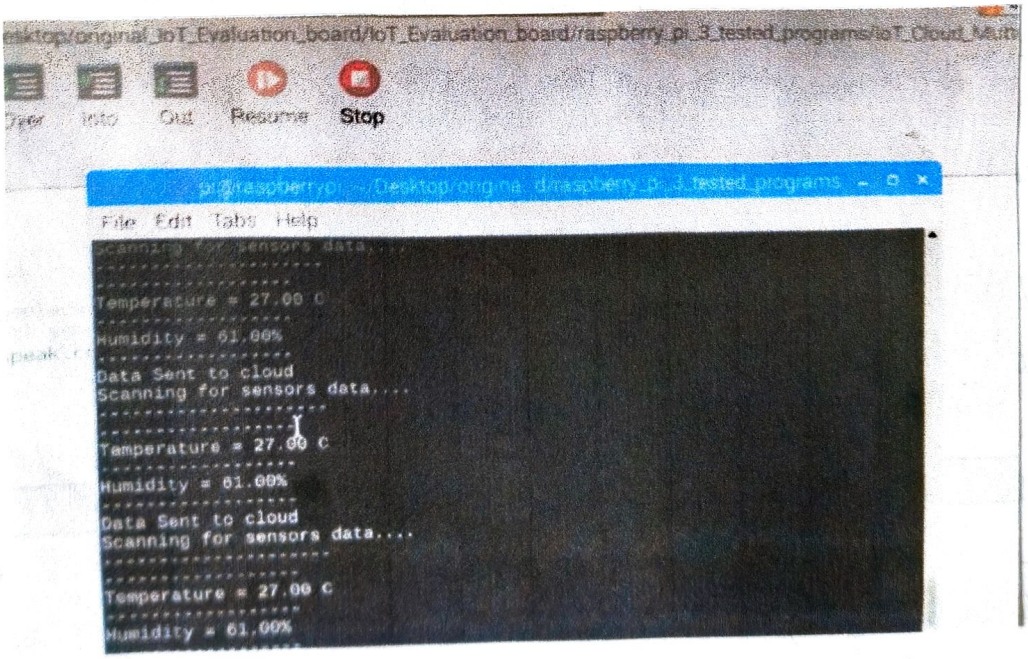
v. Program execution:
* Save and execute the program.
* Observe the terminal output showing live sensor readings and messages.

vi. Cloud Monitoring:
* Open ThingSpeak channel dashboard in a web browser.
* Verify that temperature and humidity graphs are updating in real-time.

vii. Observation and Verification:
* Observe the variations in readings both on the terminal and on ThingSpeak graphs.

```
        time.sleep(1)
except:
        print ('exception')
        break
```





d d m m y y y y

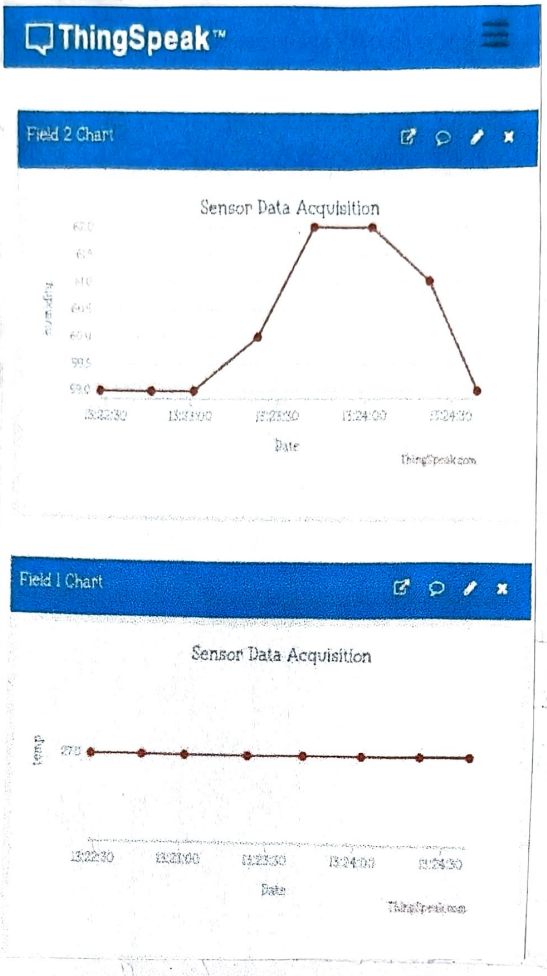
Result:

The on-board Raspberry Pi is successfully interfaced with the DHT-11 sensor on the IoT Dev board. The measured temperature & humidity data were displayed on the Raspberry Pi terminal and transmitted to the ThingSpeak cloud where the readings were visualised as real-time graphs. demonstrating successful IoT cloud integration.

(5)

15/10/25

Thingspeak graph



Experiment - 06

Aim: To interface an audio sensor with an STM32 nucleo board and develop an embedded C program using STM32 Cube IDE to:

* Read the analog sound intensity using the ADC module.
* Display the corresponding values on the serial monitor.

Components required: STM32 nucleo board, analog audio sensor module, LED, jumper wires, bread board, USB cable and STM32 Cube IDE software.

Procedure:

i. Hardware Setup:

* Connect the VCC pin of the sound sensor to 3.3 V on the STM32 board.
* Connect the GND pin to ground.
* Connect the analog output of the audio sensor to an ADC pin.

ii. STM32 Cube IDE configuration:

* Create a new STM32 project and select your Nucleo board (F411RE)
* In the pinout configuration
  - Enable ADC1 → Select input pin connected to audio sensor
  - Enable USART2 → For UART communication with PC.
  - Configure GPIOA Pin 5 as output Push-Pull for LED control.

Program:

```
#include "main.h"
#include <stdio.h>
#include <string.h>

uint16_t lvsc = 0;
char msg[20];

while (1)
{
 HAL_ADC_Start (&hadc1);
 HAL_ADC_PollForConversion (&hadc1, 20);
 lvsc = HAL_ADC_GetValue (&hadc1);
 sprintf (msg, "Sound : %.hu \r\n", lvsc);
 HAL_UART_Transmit (&huart2, uint8_t *) msg.
  strlen(msg), HAL_MAX_DELAY);

 if (lvsc < 2000)
    HAL_GPIO_WritePin (GPIOA, GPIO_PIN_5, 1);
else
   HAL_GPIO_WritePin (GPIOA, GPIO_PIN_5, 0);
HAL_Delay (2000);
}
```

d d m m y y y y

iii. Code upload and testing:

* Build the project and flash it to the STM32 board.
* Open the serial monitor via STM32 Cube IDE console.
* Observe the ADC readings displayed in real-time.

iv. Observation:

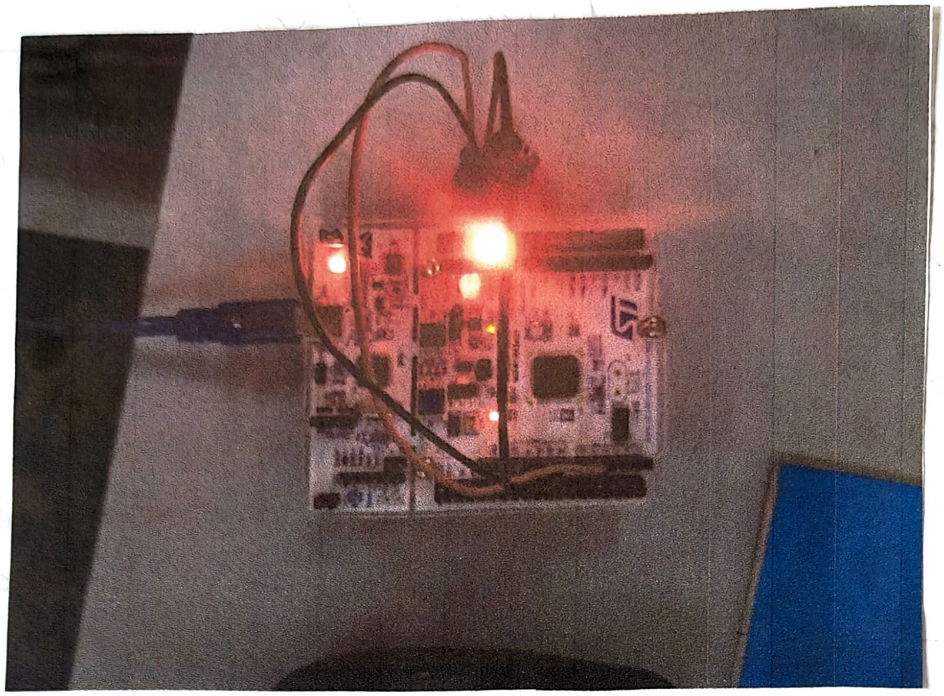
* LED ON: when sound level is below threshold -> quiet
* LED OFF: When sound level is above threshold -> loud sound detected.

Result:

The STM32 successfully reads sound intensity through the ADC & controls the LED according to the sound level. UART serial output displays real-time ADC values demonstrating analog sensor interfacing & digital output control.

(5)

15/10/25



Serial monitor

```
107         HAL_ADC_Start(&hadc1);
108         HAL_ADC_PollForConversion(&hadc1,20);
109         lux=HAL_ADC_GetValue(&hadc1);
110         sprintf(msg, "Light: %hu \r\n", lux);
111         HAL_UART_Transmit(&huart2,(uint8_t*)msg, strlen(m
112
113
114         if(lux<2000){
115             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN
116         }else{
117             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN
118         }
119         HAL_Delay(200);
120     }
121     /* USER CODE END 3 */
122 }
```

Console × Properties

ADC 1 (CONNECTED)

```
Light: 2178
Light: 2031
Light: 1962
Light: 1685
Light: 2135
Light: 2011
Light: 2010
Light: 1991
```

22/02/2025

Experiment - 07

Aim: To interface Ultrasonic sensor with the Raspberry Pi integrated on the IoT development board.

Components required: IoT development board, Raspberry Pi, Ultrasonic sensor (on-board), ethernet, HDMI & display, keyboard, mouse and python with libraries.

Procedure:

i. Hardware Setup:

* Connect the HDMI cable from the Raspberry Pi to a display monitor.
* Connect the mouse and keyboard to the USB port of Raspberry Pi.
* The ultrasonic sensor is pre-connected to one of the Raspberry Pi GPIO pin.

ii. Python Development:

* Open Thonny IDE on the raspberry Pi display.
* Create a new file named IoT_ultrasonic.py.
* Define the API key, base URL and sensor type.

iii. Program logic:

* Continuously read the measurements.
* Display sensor reading on raspberry Pi terminal.
* Send the readings to the cloud using URL.

Program:

```
import time
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
TRIG = 19
ECHO = 26
print("Distance measurement in progress")
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
while True:
    GPIO.output(TRIG, False)
    print("Waiting for sensor to settle")
    time.sleep(2)
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)
while GPIO.input(ECHO) == 0:
    pulse_end = time.time()
pulse_duration = pulse_end - pulse_start
distance = pulse_duration * 17150
distance = round(distance, 2)

if distance > 2 and distance < 400:
    print("Distance:", distance - 0.5, "cm")
else:
    print("Out of Range")
```

d d m m y y y y

iv. Program execution:

* Save & execute the program.
* Observe the terminal o/p showing sensor readings

v. Observation and verification:

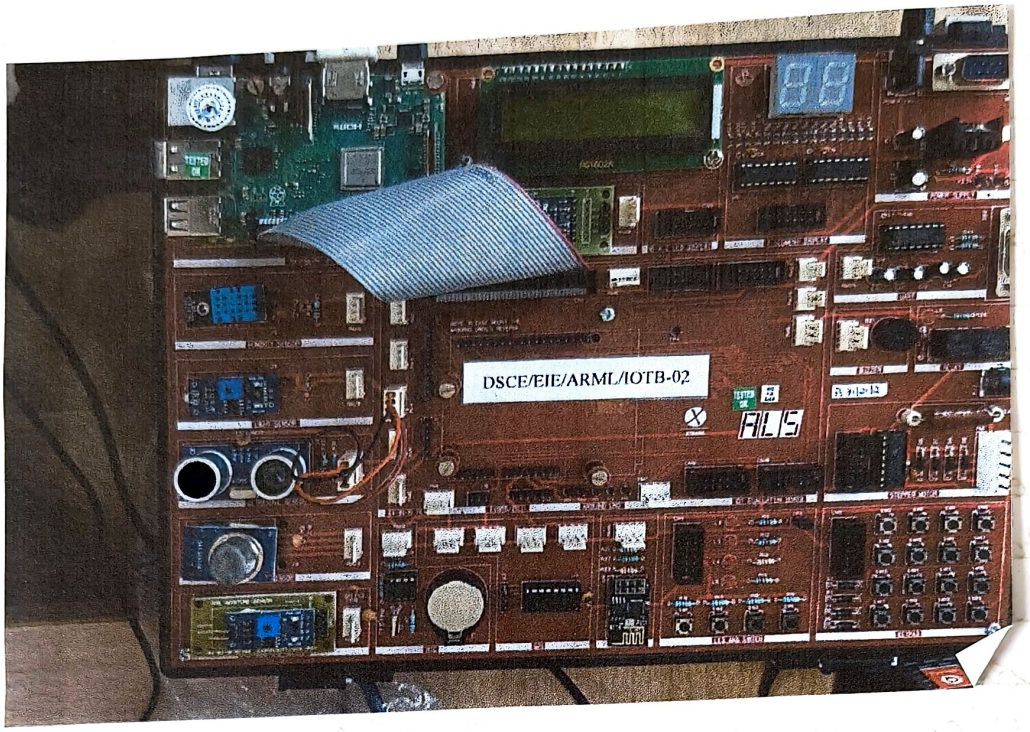
* Change the surrounding measurement slightly.
* Observe the variation in readings both on the terminal & the window.

Result:

The on-board Raspberry Pi is successfully interfaced with the ultrasonic sensor on the IoT development board. The measured data were displayed on the terminal demonstrating successful IoT cloud-integration.

5

22/10/3



Terminal

```
Shell
Distance: 2.57 cm
Waitng For Sensor To Settle
Distance: 2.58 cm
Waitng For Sensor To Settle
Distance: 2.44 cm
Waitng For Sensor To Settle
Distance: 2.44 cm
Waitng For Sensor To Settle
Distance: 1.7200000000000002 cm
Waitng For Sensor To Settle
Distance: 2.41 cm
Waitng For Sensor To Settle
Distance: 2.41 cm
Waitng For Sensor To Settle
```